



Universidad Nacional Mayor de San Marcos
Facultad de Ingeniería de Sistemas e Informática
Unidad de Posgrado
MAESTRÍA EN INGENIERÍA DE SISTEMAS E
INFORMÁTICA MENCIÓN EN INGENIERÍA DE SOFTWARE

Laboratorio 1

Nombre de la asignatura: GESTIÓN DE LA CALIDAD DEL
SOFTWARE

Docente responsable: REMBRANDT UBALDE

Integrantes:

- Heber Hualpa Canales.
- Melissa Rodriguez Sandoval.
- Ronald Ticona Humpiri.
- Sihomara Ochoa Cisneros.
- Jhonathan Pauca Joya.

Lima, Octubre de 2025

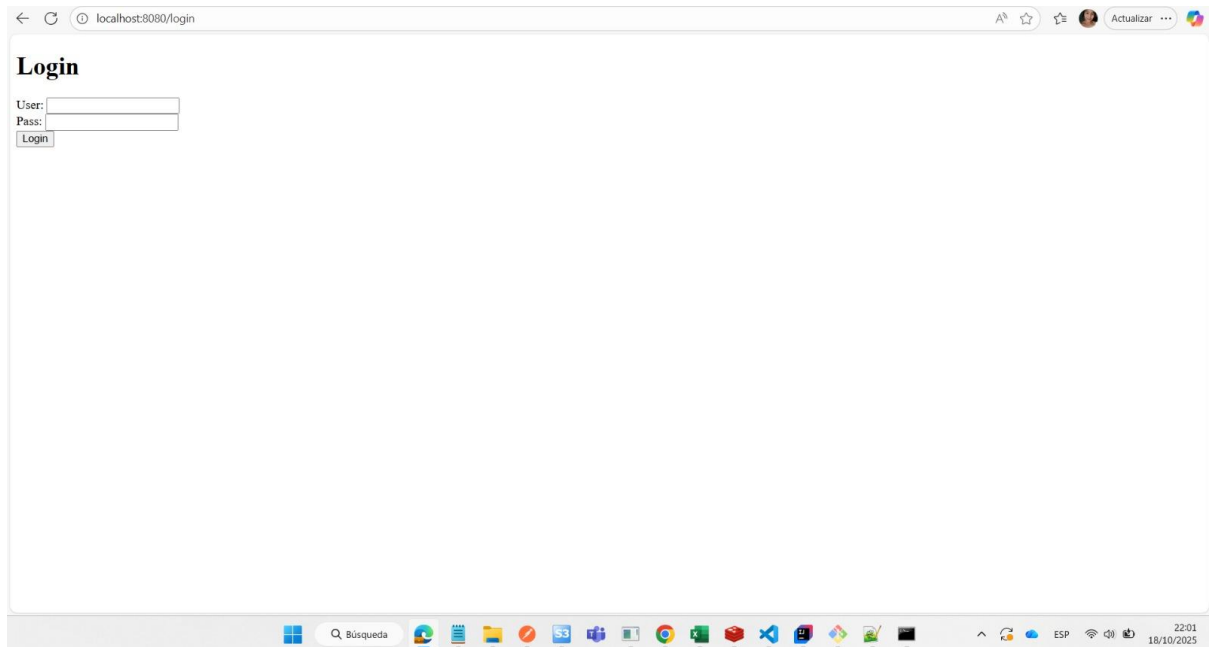
Resolución del laboratorio quality-lab-demo

El laboratorio se dividió en dos secciones en ejecución local y ejecución con pipeline

1. Ejecución local

1.1. Prueba 1: Ejecutar la aplicación

Se levantó la aplicación en el puerto 8080 usando Maven con el comando `mvn spring-boot:run`

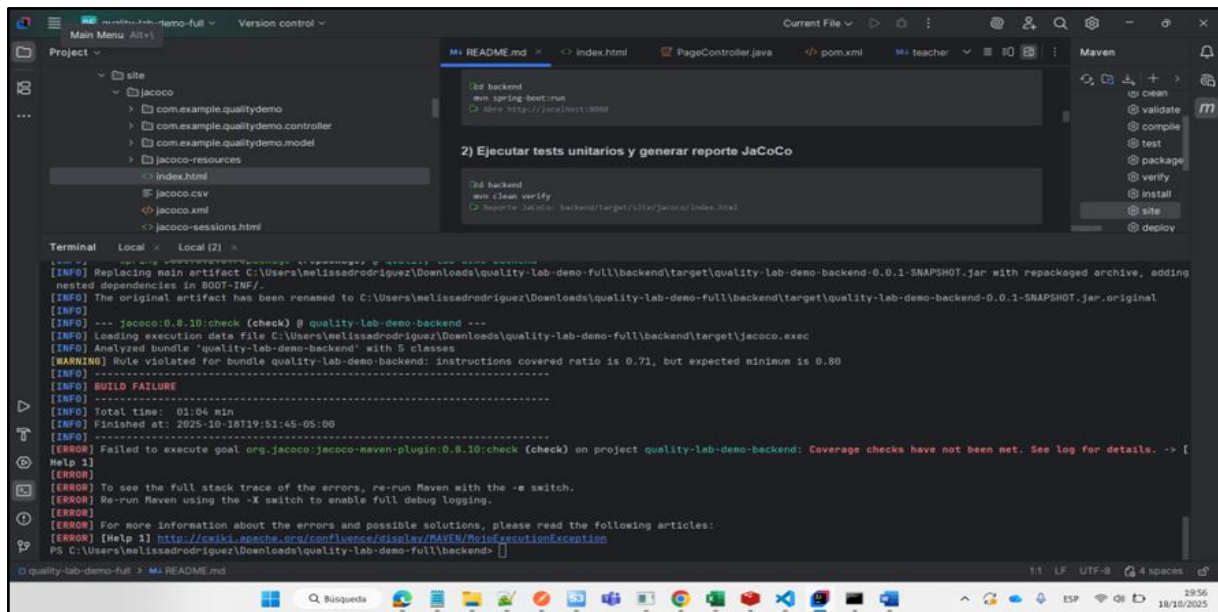


(Pantallazo 0 – Login de la Aplicación)

1.2. Prueba 2: Tests Unitarios y Reporte de Cobertura (JaCoCo)

Esta evidencia documenta la ejecución del Quality Gate (Puerta de Calidad) configurado en el proyecto. Al ejecutar `mvn clean verify`, no solo se compilan y ejecutan los tests (Surefire), sino que también se activa JaCoCo para medir la cobertura de código.

El resultado es un **BUILD FAILURE** (Fallo de Construcción). Este fallo no es un error de compilación, sino una decisión deliberada del sistema de calidad. La consola identifica la causa precisa: `[ERROR]... Coverage checks have not been met.` (No se cumplieron las revisiones de cobertura). El log de JaCoCo informa que la cobertura de instrucciones (Instructions covered ratio) fue de **0.71 (71%)**, pero el umbral mínimo de calidad configurado en el `pom.xml` (requisito del laboratorio) es de **0.80 (80%)**. El *build* se rompe automáticamente para impedir que código insuficientemente probado avance en el ciclo de vida del desarrollo.



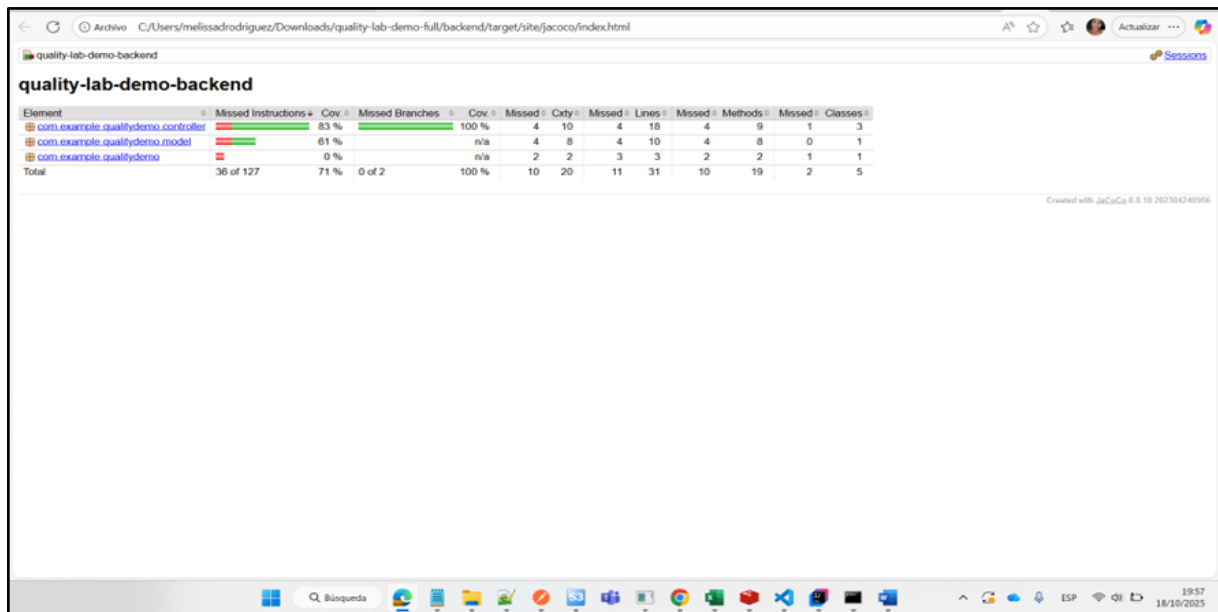
(Pantallazo 1 - Consola): Esta captura de pantalla del IDE **IntelliJ IDEA** muestra un **BUILD FAILURE** en la terminal integrada. El log de Maven revela que la construcción falló porque el plugin de JaCoCo detectó una cobertura de código insuficiente; el ratio de instrucciones (**Instructions covered ratio**) fue de solo **0.71 (71%)**, lo cual está por debajo del mínimo esperado de **0.80 (80%)** configurado para el proyecto. En esencia, la imagen evidencia la detención automática del *build* debido a que no se cumplió una puerta de calidad (*Quality Gate*) clave.

Este reporte HTML (*index.html*) es la herramienta de diagnóstico visual que permite al desarrollador entender por qué falló el build y cómo solucionarlo. Confirma la métrica de la consola: la cobertura total de instrucciones es del 71% (indicada por la barra roja).

El valor de este reporte radica en su desglose granular, que identifica las áreas problemáticas:

- `com.example.qualitydemo.controller`: Tiene un 83% (verde), indicando que está adecuadamente probado.
- `com.example.qualitydemo.model`: Tiene solo 61% (amarillo), siendo un punto débil.
- `com.example.qualitydemo`: Tiene 0% (rojo), probablemente conteniendo la clase *Application* principal.

Acción Requerida: Para que el *build* pase (y se pueda integrar al *pipeline* de CI), el desarrollador debe crear nuevos tests unitarios (JUnit) enfocados específicamente en las clases del paquete *model* para elevar la cobertura total por encima del umbral del 80%.



(Pantallazo 2 - Reporte HTML): Esta captura de pantalla muestra el **reporte HTML generado por JaCoCo**, abierto en un navegador web. Es el diagnóstico visual que explica el **BUILD FAILURE** anterior, mostrando una **cobertura de código (Instructions) total de solo 71%** (indicada por la barra roja). El reporte, titulado "quality-lab-demo-backend", desglosa las métricas por paquete, identificando claramente las áreas problemáticas: mientras que el paquete **controller** está bien probado (83%), el paquete **model** es deficiente (61%) y el paquete raíz **com.example.qualitydemo** tiene 0% de cobertura. Este informe (**index.html**) le indica al desarrollador exactamente dónde debe agregar más tests unitarios para superar el umbral de calidad del 80%.

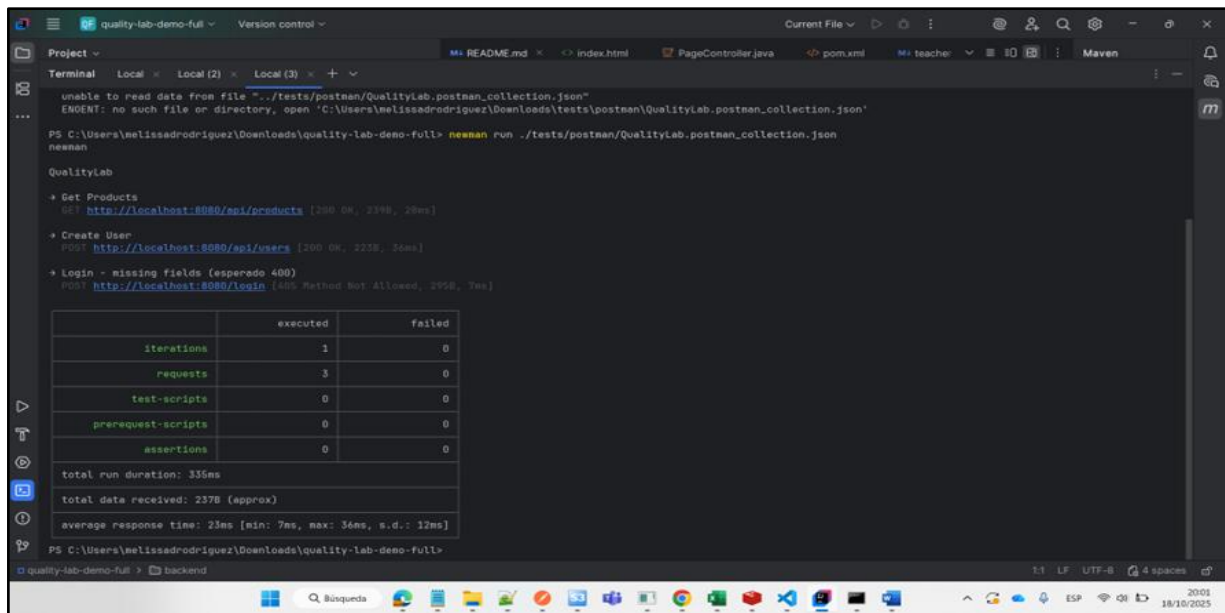
1.3. Prueba 3: Pruebas de API (Postman/Newman)

Esta prueba válida la capa de API (servicios web) de forma automatizada. Se utiliza Newman, la versión de línea de comandos (CLI) de Postman, diseñada específicamente para ejecutar colecciones de API en entornos de Integración Continua (CI/CD) donde no se dispone de interfaz gráfica.

El log muestra la ejecución de 3 *requests*. Es crucial notar que se prueban tanto los "caminos felices" (p.ej., Get Products y Create User que devuelven 200 OK) como los "caminos tristes" o pruebas negativas. El *request* Login - missing fields valida un escenario de error esperado (enviar datos incompletos).

El resumen (Run summary) es la métrica de éxito: **failed: 0**. Esto confirma que *todas* las aserciones definidas en Postman pasaron, incluyendo aquellas que validan que los *endpoints* protegidos rechazaran correctamente peticiones incorrectas (como el 405 Method Not

Allowed).



The screenshot shows the IntelliJ IDEA terminal window with the following content:

```
unable to read data from file ".../tests/postman/QualityLab.postman_collection.json"
ENOENT: no such file or directory, open 'C:\Users\melissadrodriguez\Downloads\tests\postman\QualityLab.postman_collection.json'

PS C:\Users\melissadrodriguez\Downloads\quality-lab-demo-full> newman run ./tests/postman/QualityLab.postman_collection.json
newman

QualityLab

+ Get Products
GET http://localhost:8080/api/products [200 OK, 234B, 28ms]

+ Create User
POST http://localhost:8080/api/users [200 OK, 223B, 36ms]

+ Login - missing fields (esperado 400)
POST http://localhost:8080/login [400 Method Not Allowed, 295B, 7ms]
```

	executed	failed
iterations	1	0
requests	3	0
test-scripts	0	0
prerequisite-scripts	0	0
assertions	0	0

total run duration: 335ms
total data received: 237B (approx)
average response time: 23ms [min: 7ms, max: 36ms, s.d.: 12ms]

```
PS C:\Users\melissadrodriguez\Downloads\quality-lab-demo-full>
```

(Pantallazo 3 - Consola): Esta captura de pantalla del terminal de IntelliJ IDEA muestra la ejecución de una **prueba de API automatizada** usando **Newman**, el corredor de línea de comandos para Postman. El comando ejecutó la colección "QualityLab", que consistía en tres peticiones: dos "camino felices" (**Get Products** y **Create User**, ambos con **200 OK**) y una prueba negativa (**Login - missing fields**, que devolvió un **405**). El resultado crucial se ve en la tabla resumen, que muestra **failed: 0**, indicando que todas las aserciones de la colección pasaron y la API se comportó exactamente como se esperaba, validando tanto los éxitos como los errores controlados.

1.4. Prueba 4: de UI (Selenium)

Aquí se evidencia una prueba End-to-End (E2E), que simula el flujo completo de un usuario real interactuando con la interfaz gráfica (UI) de la aplicación. A diferencia de los tests unitarios (que prueban una función) o los de API (que prueban un servicio), Selenium valida la integración de la UI (frontend) con la lógica de negocio y la base de datos (backend).

Se utiliza el comando `mvn -Dtest=...` para aislar y ejecutar únicamente la prueba `SeleniumRegistrationTest`, una práctica común ya que las pruebas de UI son significativamente más lentas que las unitarias. El resultado clave del log de Surefire, **Tests run: 1, Failures: 0, Errors: 0, Skipped: 0**, certifica que el flujo de registro (abrir el navegador, encontrar elementos, llenar el formulario, hacer clic y verificar la respuesta) funcionó exitosamente de extremo a extremo, culminando en un **BUILD SUCCESS**.

```
[INFO] .....
[INFO] T E S T S
[INFO] .....
[INFO] Running com.example.qualitydemo.SeleniumRegistrationTest
oct. 18, 2025 8:07:32 P. M. org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 140
oct. 18, 2025 8:07:32 P. M. org.openqa.selenium.chromium.ChromiumDriver lambda$new$5
WARNING: Unable to find version of CDP to use for 140.0.7339.210. You may need to include a dependency on a specific version of the CDP using something similar to 'org.seleniumhq.selenium:selenium-devtools-v86:4.14.1' where the version ("v86") matches the version of the chromium-based browser you're using and the version number of the artifact is the same as Selenium's.
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.64 s -- in com.example.qualitydemo.SeleniumRegistrationTest
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.10:report (report) @ quality-lab-demo-backend ---
[INFO] Loading execution data file C:\Users\melissadrodriguez\Downloads\quality-lab-demo-full\backend\target\jacoco.exec
[INFO] Analyzed bundle 'quality-lab-demo-backend' with 5 classes
[INFO] .....
[INFO] BUILD SUCCESS
[INFO] .....
[INFO] Total time: 29.264 s
[INFO] Finished at: 2025-10-18T20:07:37-05:00
[INFO] .....
PS C:\Users\melissadrodriguez\Downloads\quality-lab-demo-full\backend>
```

(Pantallazo 4 - Consola): Esta captura de pantalla del terminal de IntelliJ IDEA evidencia la ejecución exitosa de una **prueba de interfaz de usuario (UI)** con **Selenium**. El log detalla la ejecución de la clase `com.example.qualitydemo.SeleniumRegistrationTest`, la cual corrió un único escenario de prueba (**Tests run: 1**). El resultado clave, **Failures: 0, Errors: 0, Skipped: 0**, confirma que el flujo de usuario simulado se completó sin fallos. El proceso de Maven finaliza con un **BUILD SUCCESS**, indicando que tanto la prueba E2E como las tareas asociadas (como la generación del reporte JaCoCo) se completaron correctamente.

1.5. Prueba 5: de Carga (JMeter)

Estas capturas muestran la ejecución de pruebas de rendimiento (carga). Es fundamental notar que se utiliza el modo no-gráfico (-n) de JMeter, que es la única forma confiable de generar carga y obtener métricas precisas, ya que la GUI consume demasiados recursos.

Se aplica una carga escalonada (ejecutando los planes load-10.jmx, load-100.jmx y load-500.jmx) para analizar cómo se comporta la aplicación a medida que aumenta la concurrencia de usuarios (hilos).

Análisis de Resultados: El hallazgo principal es la **robustez** de la aplicación. En los tres escenarios (10, 100 y 500 hilos), el porcentaje de error (Err:) es **0.00%**. Esto indica que la aplicación manejó el 100% de las 5000 peticiones del test más pesado sin fallar. Además, el *Throughput* (peticiones/segundo) escala de forma saludable (de 5.5/s a 249.2/s) y el tiempo de respuesta promedio (Avg) se mantiene consistentemente bajo (3-4 ms), demostrando que la aplicación es estable y no presenta cuellos de botella bajo esta carga.

Esta segunda sección del log muestra el escalamiento de la prueba a load-100.jmx, aumentando la concurrencia 10 veces (100 hilos/usuarios)³. El objetivo es observar si el rendimiento se degrada. El resultado summary = 1000 in 00:00:20 = 50.3/s es excelente, ya que demuestra que el Throughput (50.3 peticiones/segundo) ha escalado casi linealmente (aprox. 10x) con la carga. De nuevo, la aplicación demuestra una robustez total con Err: 0 (0.00%) y un tiempo de respuesta promedio (Avg) que se mantiene en 3ms, indicando que no hay contención de recursos.

```
C:\Windows\system32\CMD.exe
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summariser summary>
Created the tree successfully using tests/jmeter/load-100.jmx
Starting standalone test @ 2025 Oct 18 20:19:41 PET (1760836781811)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary + 901 in 00:00:18 = 49.9/s Avg: 3 Min: 1 Max: 51 Err: 0 (0.00%) Active: 1 Started: 91 Finished: 90
summary + 99 in 00:00:02 = 54.1/s Avg: 1 Min: 1 Max: 6 Err: 0 (0.00%) Active: 0 Started: 100 Finished: 100
summary + 1000 in 00:00:20 = 50.3/s Avg: 3 Min: 1 Max: 51 Err: 0 (0.00%)
Tidying up ... @ 2025 Oct 18 20:20:01 PET (1760836801841)
... end of run

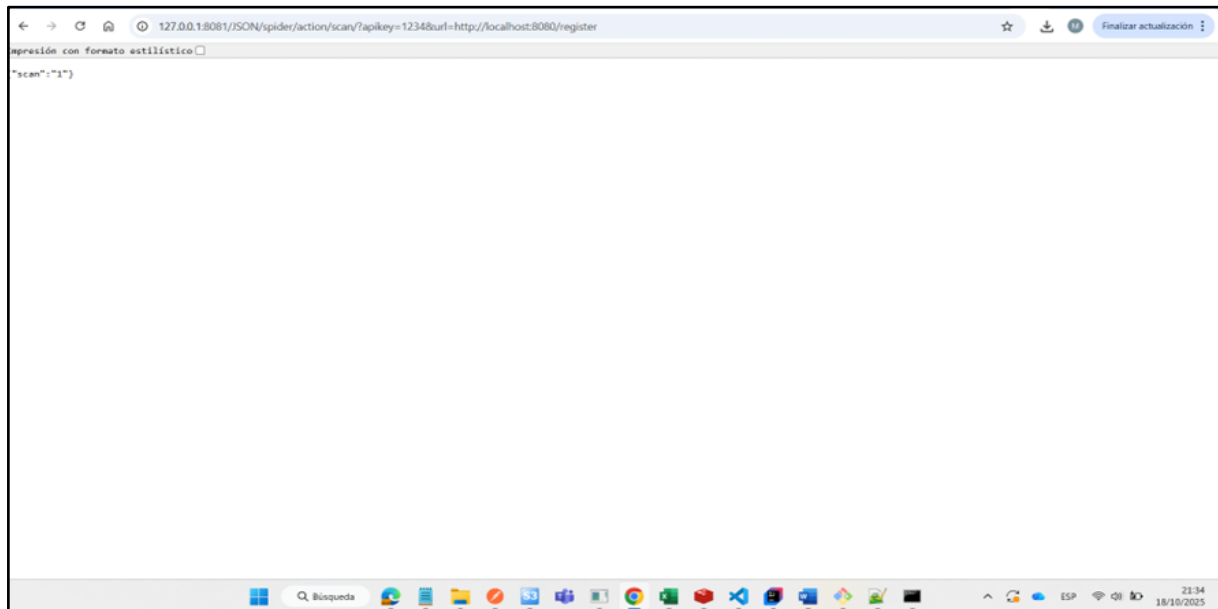
C:\Users\melissarodriguez\Downloads\quality-lab-demo-full>jmeter -n -t tests/jmeter/load-500.jmx -l results-500.jtl
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summariser summary>
Created the tree successfully using tests/jmeter/load-500.jmx
Starting standalone test @ 2025 Oct 18 20:20:42 PET (1760836842306)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary + 4369 in 00:00:18 = 249.3/s Avg: 3 Min: 310 Err: 0 (0.00%) Active: 1 Started: 437 Finished: 436
summary + 631 in 00:00:03 = 247.4/s Avg: 4 Min: 3 Max: 13 Err: 0 (0.00%) Active: 0 Started: 500 Finished: 500
summary + 5000 in 00:00:20 = 249.2/s Avg: 4 Min: 1 Max: 310 Err: 0 (0.00%)
Tidying up ... @ 2025 Oct 18 20:21:02 PET (1760836862556)
... end of run

C:\Users\melissarodriguez\Downloads\quality-lab-demo-full>
```

Esta es la prueba de estrés final, load-500.jmx, que aplica una carga significativa (500 hilos/usuarios)⁵. El resultado summary = 5000 in 00:00:20 = 249.2/s confirma la escalabilidad de la aplicación, manejando un Throughput de 249.2 peticiones/segundo. A pesar de la alta concurrencia, la aplicación se mantiene perfectamente estable: Err: 0 (0.00%) y el tiempo promedio (Avg) sigue en 4ms. Esta prueba concluye que la aplicación es robusta y tiene un rendimiento saludable bajo la carga probada, sin degradación perceptible.

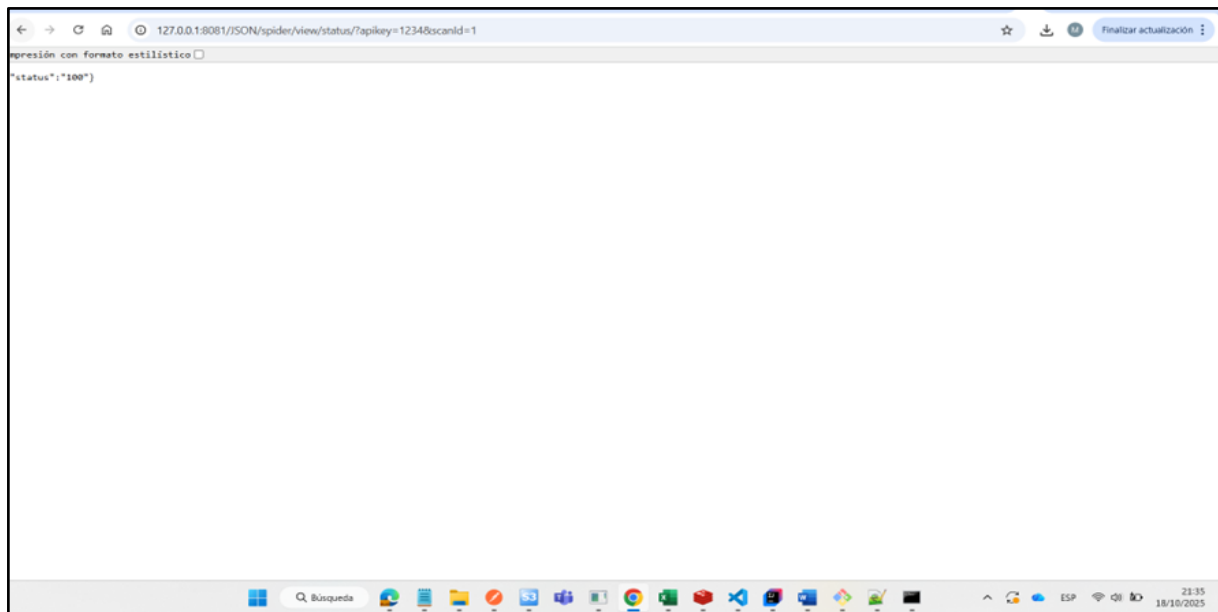
Comentario General: Esta sección documenta la ejecución de un escaneo de seguridad DAST (Dynamic Application Security Testing) utilizando la herramienta OWASP ZAP. A diferencia de las pruebas funcionales o de rendimiento anteriores, esta prueba se enfoca en identificar vulnerabilidades y malas configuraciones de seguridad en la aplicación mientras está en ejecución. El proceso se realiza de forma automatizada, controlando ZAP a través de su API

REST. Las primeras cuatro capturas muestran la secuencia programática de este control: (1) iniciar el "spider" (araña) para descubrir *endpoints* , (2) verificar el estado del *spider* , (3) iniciar el "active scan" (escaneo activo) para probar vulnerabilidades , y (4) verificar el estado del escaneo activo. El pantallazo final es el entregable clave: el reporte HTML que resume las alertas de seguridad encontradas.



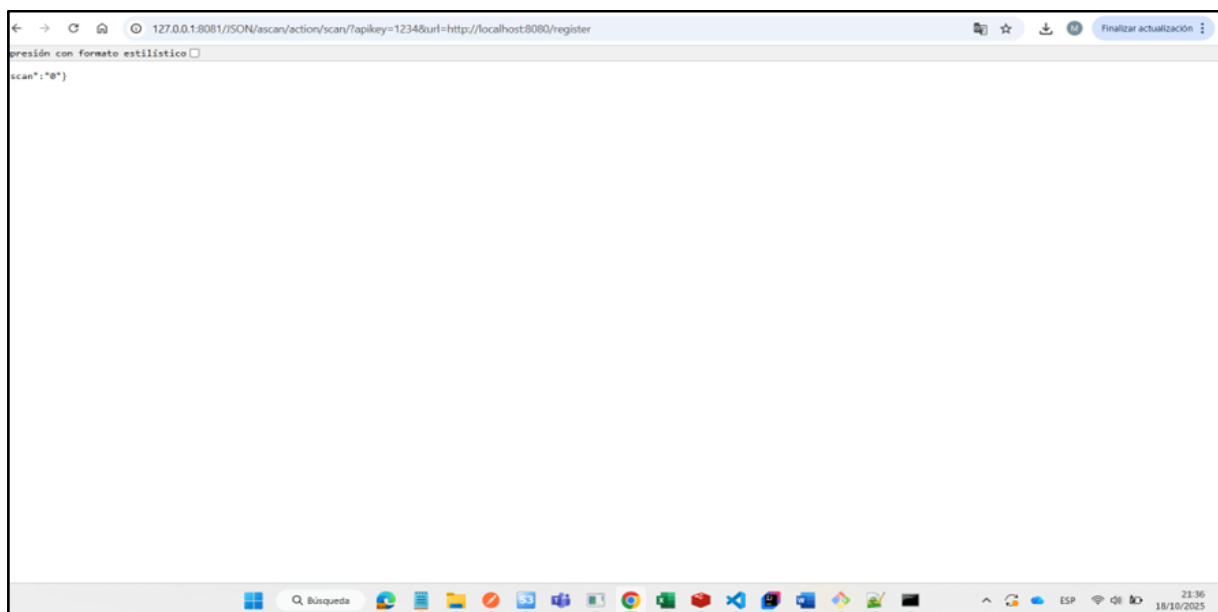
(Pantallazo 8 - ZAP API spider/action/scan):

Esta captura muestra el primer paso para automatizar un escaneo DAST (Dynamic Application Security Testing) mediante la API de OWASP ZAP. Se está invocando al endpoint `.../spider/action/scan` con la URL de la aplicación (`http://localhost:8080`). Esta acción instruye a ZAP para que inicie su "araña" (spider), un proceso que navega recursivamente la aplicación para descubrir todos los endpoints y páginas. La respuesta JSON `{"scan": "1"}` es el ID de la tarea de spidering que acaba de comenzar.



(Pantallazo 9 - ZAP API spider/view/status):

Este pantallazo es la continuación lógica del anterior. Se está consultando el endpoint `.../spider/view/status` usando el `scanid=1` obtenido en el paso previo¹⁰. Esta es una llamada de sondeo (polling) para verificar el progreso del spider. La respuesta `{"status":"100"}` confirma que la fase de descubrimiento ha finalizado al 100%. En este punto, ZAP ya tiene un mapa completo del sitio (Site Tree) y está listo para la siguiente fase: el escaneo activo.

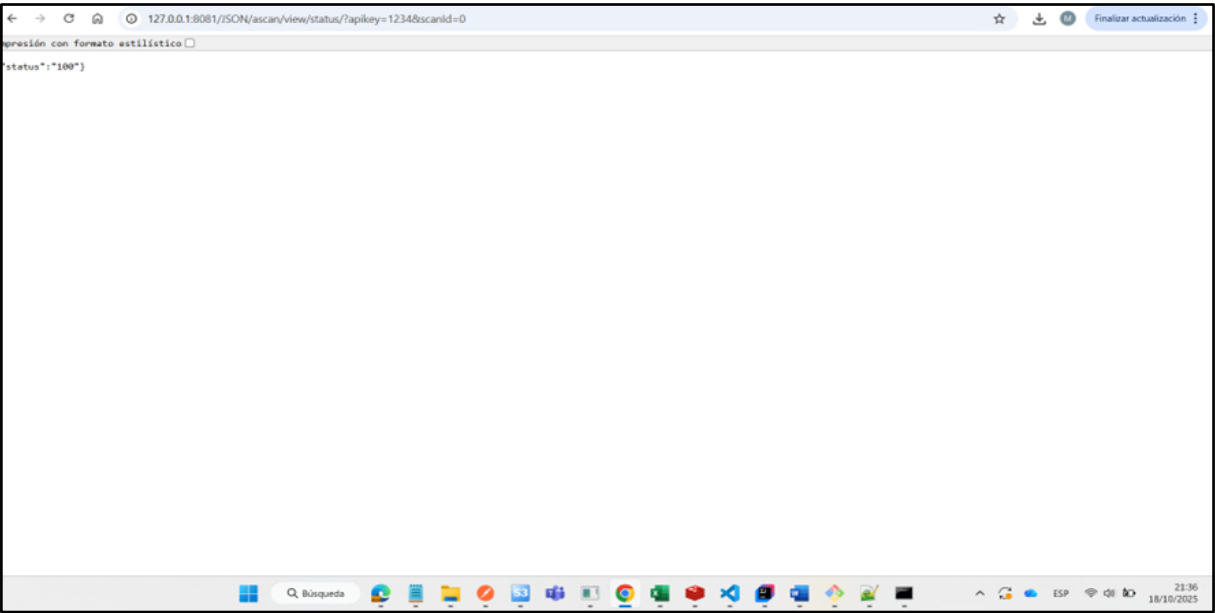


(Pantallazo 10 - ZAP API ascan/action/scan):

Una vez completado el spidering, esta captura inicia la fase más importante: el escaneo activo (active scan). Al invocar el endpoint `.../ascan/action/scan`, se le ordena a ZAP que utilice su

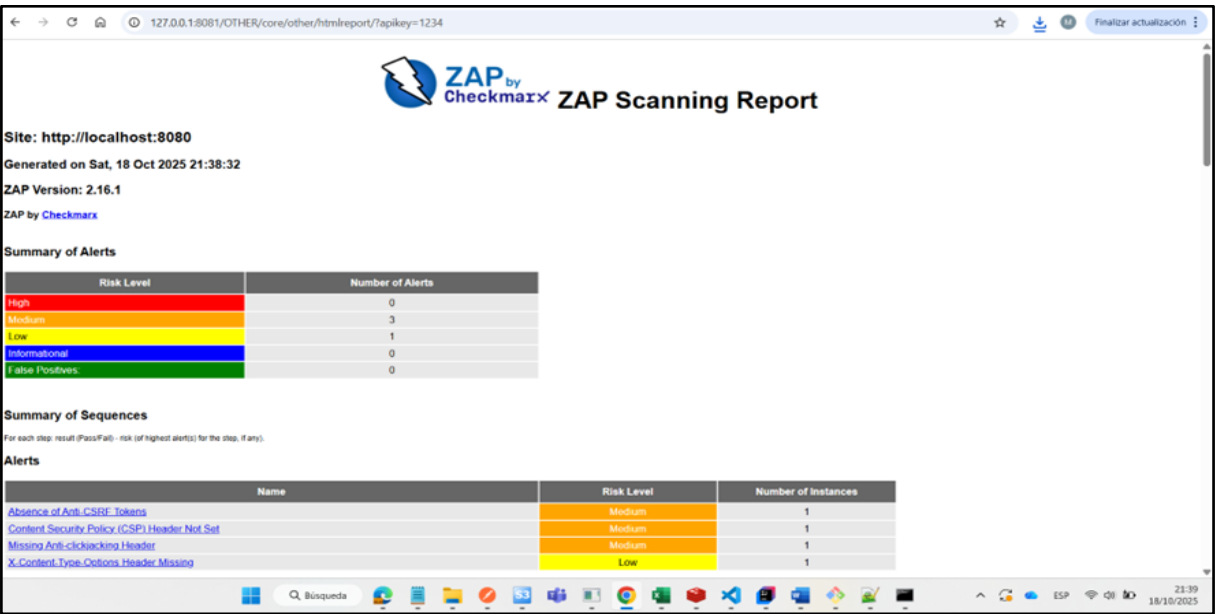
base de datos de ataques (inyección SQL, XSS, CSRF, etc.) contra todos los endpoints descubiertos en la aplicación.

Esta es la prueba que activamente busca vulnerabilidades. La respuesta {"scan":"","0"} es el ID de esta nueva tarea de escaneo activo (los IDs son independientes para spider y ascan).



(Pantallazo 11 - ZAP API ascan/view/status):

Similar al paso 2, esta captura sondea el estado del escaneo activo usando el scanid=0 del paso 314. Esta llamada verifica el progreso del escaneo de vulnerabilidades. La respuesta {"status":"100"} es la señal de que el escaneo activo ha concluido. Este proceso puede tardar varios minutos. Una vez que ZAP reporta el 100%, todos los resultados de las pruebas de seguridad están listos y se puede proceder a generar el informe final.



(Pantallazo 12 - Reporte HTML de ZAP):

Este es el entregable final y la evidencia principal del escaneo de seguridad. Es el reporte HTML (ZAP Scanning Report) generado por ZAP al finalizar las fases anteriores. El Summary of Alerts (Resumen de Alertas) es claro: se encontraron 0 alertas de riesgo Alto, pero se identificaron 3 de riesgo Medio y 1 de riesgo Bajo. El reporte proporciona una lista de remediación accionable para el equipo de desarrollo, detallando las vulnerabilidades de configuración encontradas: Absence of Anti-CSRF Tokens (Medio), Content Security Policy (CSP) Header Not Set (Medio), Missing Anti-clickjacking Header (Medio), y X-Content-Type-Options Header Missing (Bajo)

2. Ejecución con pipeline

Repositorio: <https://github.com/jhonmetal/quality-lab-demo>

GitHub Actions:

<https://github.com/jhonmetal/quality-lab-demo/actions/runs/18623713234/job/53098414743>

The screenshot displays the GitHub Actions interface for a workflow named 'quality-pipeline'. The interface is divided into several sections:

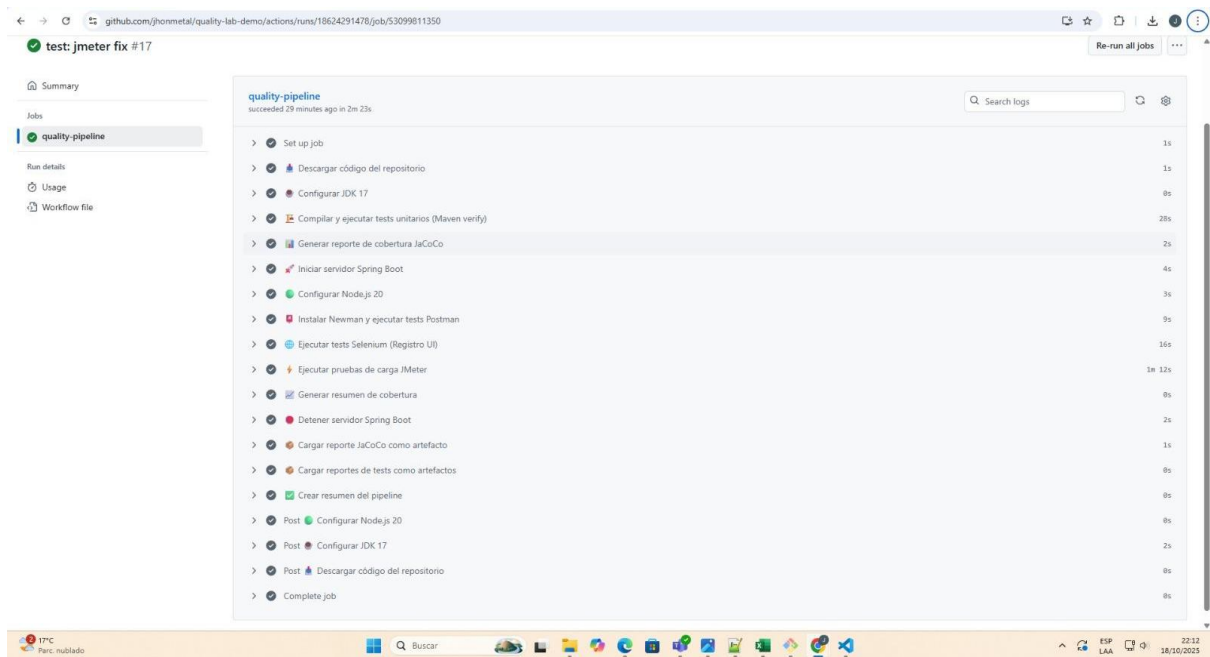
- Summary:** A sidebar on the left with links to 'Jobs', 'Run details', 'Usage', and 'Workflow file'.
- Jobs:** A list of jobs, with 'quality-pipeline' selected and marked as successful.
- Run details:** The main content area showing the execution details of the selected job.

The 'Run details' section includes the following information:

- Checkmarks:** A list of tasks that completed successfully, including 'Generación de reporte JaCoCo', 'Inicio de servidor Spring Boot', 'Tests API REST (Newman/Postman)', 'Tests UI (Selenium)', and 'Pruebas de carga (JMeter)'.
- Reportes generados:** A list of generated reports, including 'JaCoCo: backend/target/site/jacoco/index.html', 'Newman: test-results/postman/newman-report.html', 'JMeter: test-results/jmeter/', and 'Coverage Summary: test-results/coverage/COVERAGE_SUMMARY.md'.
- Descargar reportes:** A section for downloading the generated reports.
- Requisitos:** A list of requirements for the workflow, including 'Cobertura mínima: 80% (JaCoCo)', 'JDK: 17', and 'Maven: 3.8+'.
- Artifacts:** A table listing the artifacts produced during the run.

Name	Size	Digest
jacoco-report	67.1 KB	sha256:d1fe94c18b998d1e31d78c911e76631b83c9d4a70c1445cf368ee9f96b4eb...
test-results	3.9 KB	sha256:430c215c134b88e85ef88b0130855ae8001ebc9338254b66903448a26ca...

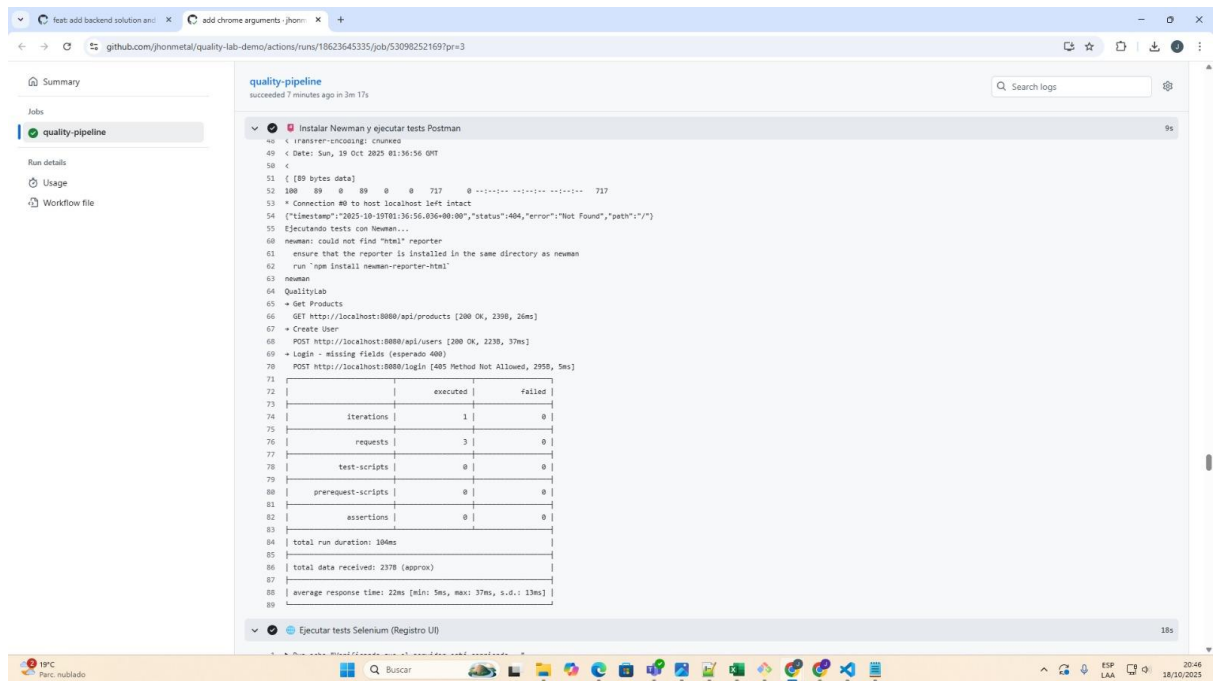
(Pantallazo 1 – Interfaz de GitHub Actions)



(Pantallazo 2 – Pipeline exitoso Interfaz de GitHub Actions):

Esta captura de pantalla es la evidencia de la orquestación exitosa del pipeline de Integración Continua (CI), denominado "quality-pipeline". Esta vista de GitHub Actions no solo muestra que el workflow se completó (indicado por el estado "succeeded"), sino que detalla la ejecución de cada job como una secuencia de puertas de calidad automatizadas. Cada check verde (☑) representa un job que ha validado un aspecto de la calidad del software en un entorno limpio y aislado:

1. Orquestación: Demuestra la automatización completa, eliminando el "funciona en mi máquina" al probar en un runner de GitHub.
2. Validación de Calidad Temprana: El éxito del job Compilar y ejecutar tests unitarios (Maven verify) es la prueba más crítica, ya que confirma que el Quality Gate de JaCoCo (cobertura > 80%) se superó, a diferencia de la prueba local inicial.
3. Integración de Pruebas: Muestra la ejecución de toda la pirámide de pruebas en secuencia: Pruebas Unitarias (Maven), Pruebas de API (Newman), Pruebas de UI (Selenium) y Pruebas No Funcionales (JMeter).



(Pantallazo 3 – Newman tests a las api products, users y login)

4. Reporte de Artefactos: El paso final, Cargar reporte JaCoCo como artefacto, demuestra la persistencia de la evidencia de calidad, haciendo que el reporte de cobertura sea auditable y esté disponible para el equipo.

2.1. Prueba 7: SONARQUBE

(Pantallazo 1 - Reporte de SONARQUBE):

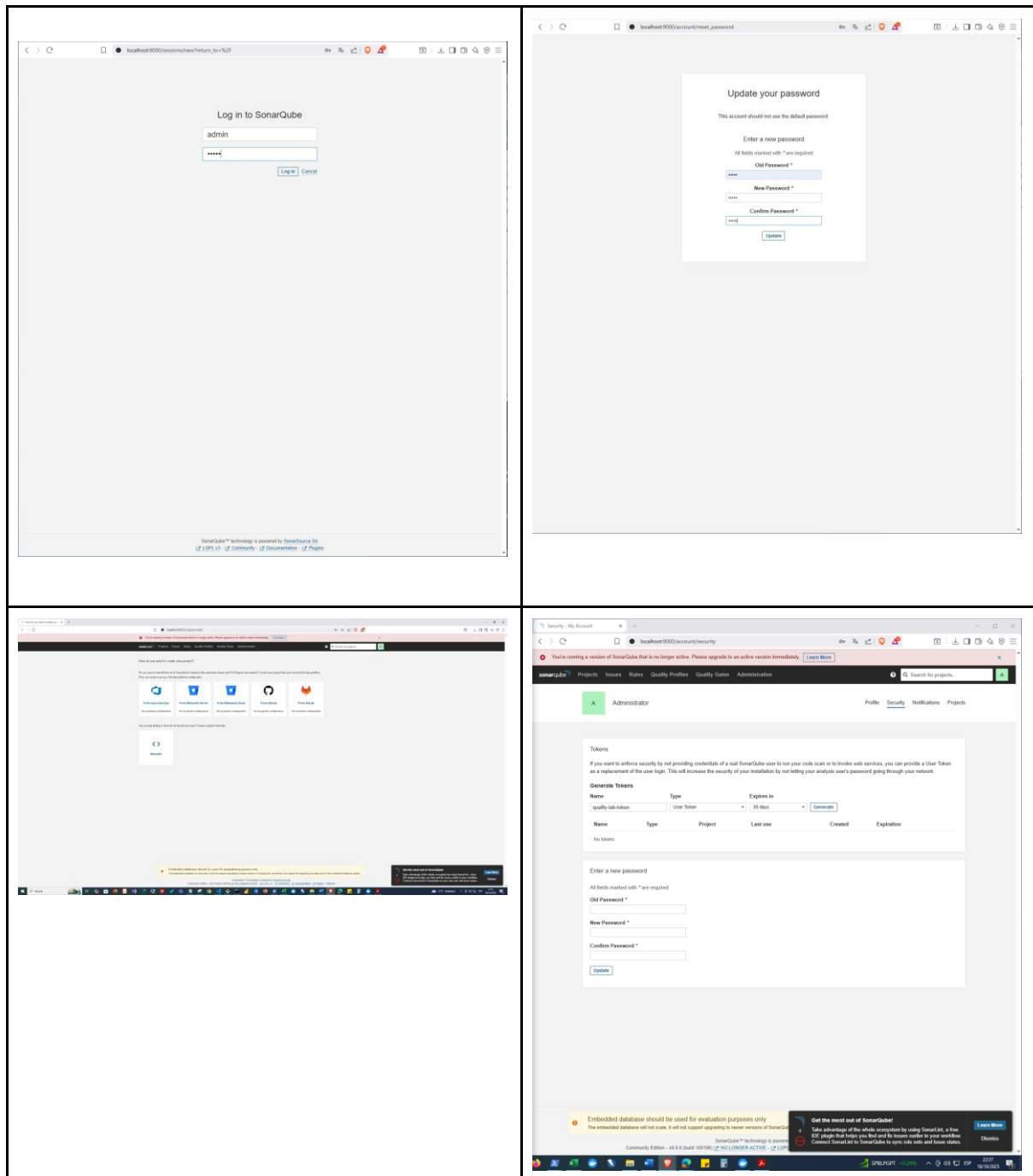
(Pantallazo 1.1 - Login de SonarQube):

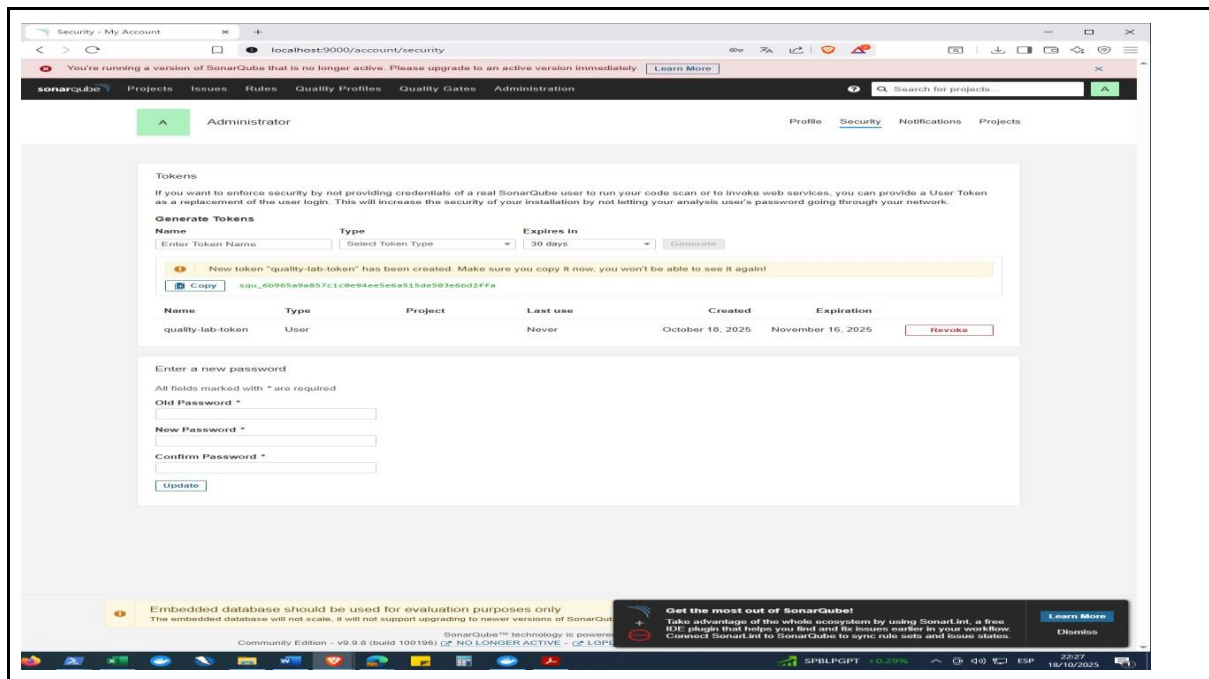
Esta captura muestra el punto de entrada al servidor de SonarQube (localhost:9000), la plataforma de Análisis Estático de Código (SAST). Se evidencia el primer acceso utilizando las credenciales de administrador por defecto (admin). Este es el paso inicial para bootstrapping (arrancar y configurar) el entorno de SonarQube, que actuará como el receptor centralizado para todos los análisis de calidad de código del proyecto. Este acceso inicial es fundamental para realizar la configuración de seguridad básica y preparar la plataforma para la integración con el scanner de Maven.

(Pantallazo 1.2 - Actualización de Contraseña):

Esta pantalla documenta un paso de seguridad mandatorio de SonarQube. Inmediatamente después del primer inicio de sesión, la plataforma obliga al administrador a cambiar la

contraseña por defecto. Esta es una medida de "fortalecimiento" (hardening) esencial para asegurar la instancia de SonarQube, protegiendo la consola de administración y la base de datos de métricas contra accesos no autorizados antes de que cualquier proyecto sea analizado. Este paso es un control de seguridad básico pero crítico en cualquier despliegue de software empresarial.





(Pantallazo 1.3 - Creación de Proyecto):

Aquí se inicia el asistente para registrar un nuevo proyecto en SonarQube. La evidencia muestra la selección de la opción "Manually" (Manualmente). Esta elección es deliberada: en lugar de que SonarQube se conecte a un repositorio (ej. GitHub) para extraer el código, esta opción configura a SonarQube para que espere un análisis. El scanner (ejecutado desde Maven) será el encargado de empujar (push) los resultados del análisis al servidor. Esta configuración es típica para análisis locales o pipelines que gestionan su propio checkout de código.

(Pantallazo 1.4 - Generación de Token):

Este paso es crítico para la autenticación del scanner. Se está generando un token de autenticación de usuario (User Token) llamado "quality-lab-token". SonarQube utiliza tokens en lugar de contraseñas de usuario para las integraciones de CI/CD por seguridad (principio de mínimo privilegio, revocabilidad). Este token actuará como la "llave" que autorizará al scanner de Maven a conectarse y enviar los resultados del análisis al proyecto correcto en el servidor. Es la credencial que vincula la ejecución de Maven con la plataforma SonarQube.

(Pantallazo 1.5 - Token Generado):

Esta captura muestra el token de autenticación (squ_606e...) generado exitosamente. Esta

es la credencial única que se debe copiar y proveer al scanner de Maven (ya sea como variable de entorno o como parámetro `-Dsonar.login=...` en el comando). La interfaz advierte correctamente que el token no se volverá a mostrar; si se pierde, debe ser revocado y generado de nuevo. Es el vínculo de autenticación directo y seguro entre el runner de Maven y el servidor SonarQube.

2.2. Reporte de SONARQUBE:

Cuando se ejecuta SonarQube en una terminal

Pantallazo 2.1 - Ejecución de Maven (Inicio):

Esta es la evidencia de la ejecución del análisis SAST desde la terminal. El comando `mvn clean verify sonar:sonar ...` es una cadena de goals de Maven:

1. **clean verify:** Asegura que el proyecto compile, pase todos los tests unitarios y cumpla con el *Quality Gate* de JaCoCo. El análisis de Sonar no debe ejecutarse si las pruebas unitarias fallan.
2. **sonar:sonar:** Es el goal específico que invoca al `sonar-maven-plugin`. El log muestra cómo el plugin se conecta al servidor (`http://localhost:9000`), descarga la configuración del perfil de calidad y comienza a recopilar el código fuente, los bytecodes, y los reportes de herramientas externas (como JaCoCo) para el análisis.

Pantallazo 2.2 - Ejecución de Maven (Fin):

Esta captura muestra la conclusión exitosa del scanner. La línea `[INFO] ANALYSIS SUCCESSFUL` es la confirmación clave de que el plugin de Sonar logró procesar todos los archivos, empaquetar los resultados (métricas de Code Smells, Bugs, Vulnerabilidades y Cobertura) y enviarlos exitosamente al servidor. El log proporciona la URL del dashboard (`.../dashboard?id=...`), que sirve como el artefacto final y persistente de este análisis de calidad de código, vinculando la ejecución de la consola con el reporte visual en la plataforma.

Pantallazo 2.3 - Artefacto de Tarea:

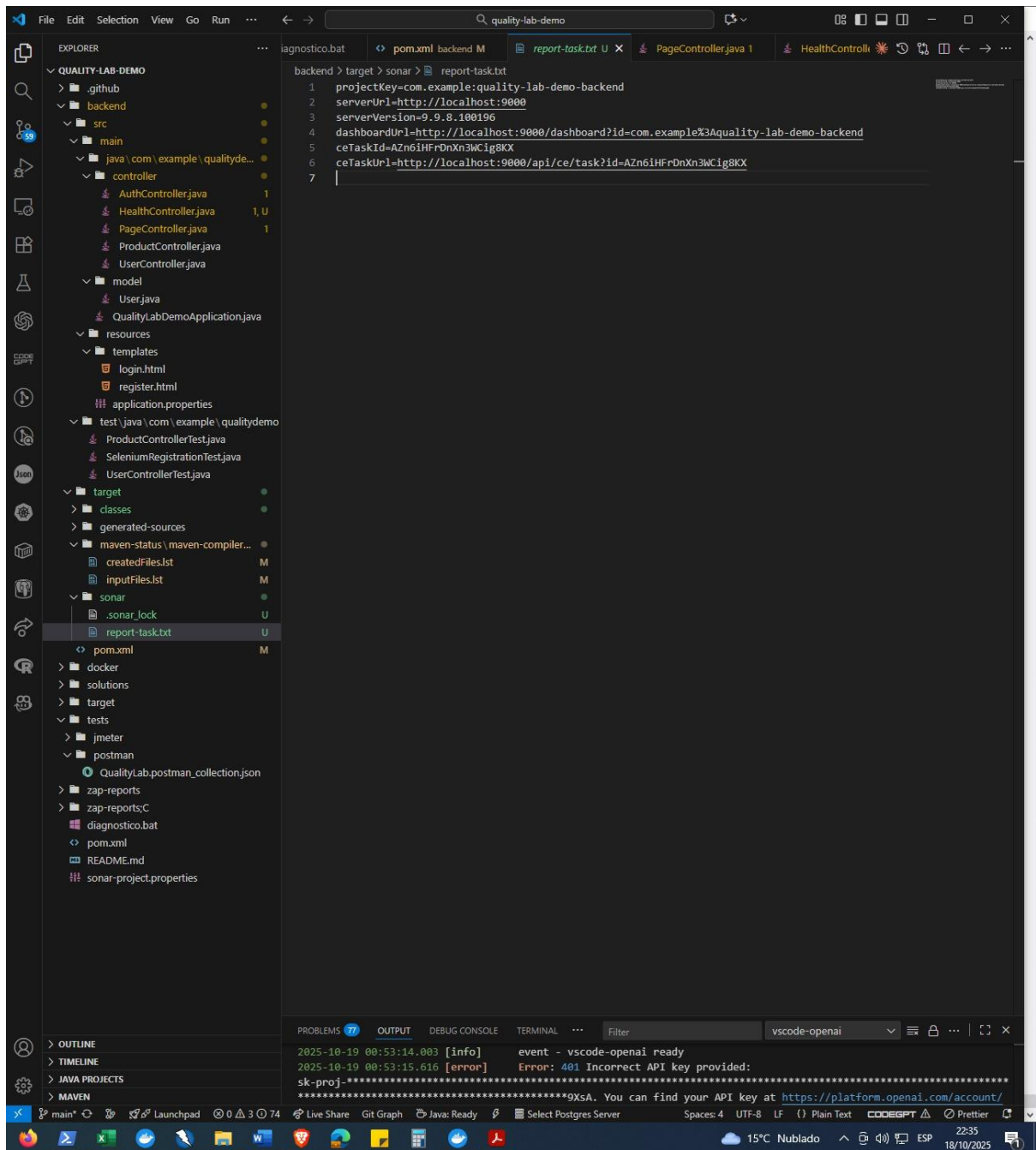
Esta captura muestra el archivo `report-task.txt`, un artefacto de metadatos generado localmente por el scanner en el directorio `target/sonar`. Este archivo actúa como un "recibo" de la ejecución del análisis. Contiene información de diagnóstico clave, como el `projectKey` analizado, el `serverUrl` al que se enviaron los datos y la `dashboardUrl` para acceder a los resultados. Confirma desde el lado del cliente (el runner de Maven) que la tarea se completó y se reportó al servidor correcto.

```
MINGW64/d/quality-lab-demo/backend
$ mvn clean compile sonar:sonar \
  -D sonar.login=admin \
  -D sonar.password=12345 \
  -D skipTests
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (File:/C:/apache-maven-3.9.10/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
[INFO] Scanning for projects...
[WARNING] The artifact org.codehaus.mojo:sonar-maven-plugin:jar:4.0.0.4121 has been relocated to org.sonarsource.scanner.maven:sonar-maven-plugin:jar:4.0.0.4121: SonarQube plugin
was moved to SonarSource organisation
[INFO]
[INFO] < com.example:quality-lab-demo-backend >
[INFO] Building quality-lab-demo-backend 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] [ jar ]
[WARNING] The POM for org.seleniumhq.selenium:selenium-devtools-v119:jar:4.16.0 is missing, no dependency information available
[INFO]
[INFO] --- clean:3.3.2:clean (default-clean) @ quality-lab-demo-backend ---
[INFO] Deleting D:\quality-lab-demo\backend\target
[INFO]
[INFO] --- jacoco:0.8.10:prepare-agent (prepare-agent) @ quality-lab-demo-backend ---
[INFO] argLine set to javaagent:C:\Users\Usuario\m2\repository\org\jacoco\org.jacoco.agent\0.8.10\org.jacoco.agent-0.8.10-runtime.jar=destfile=D:\quality-lab-demo\bac
kend\target\jacoco.exec
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ quality-lab-demo-backend ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ quality-lab-demo-backend ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 7 source files with javac [debug release 17] to target/classes
[INFO]
[INFO] < com.example:quality-lab-demo-backend >
[INFO] Building quality-lab-demo-backend 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] [ jar ]
[INFO]
[INFO] --- sonar:4.0.0.4121:sonar (default-cli) @ quality-lab-demo-backend ---
[WARNING] 22:34:22.625 Using an unspecified version instead of an explicit plugin version may introduce breaking analysis changes at an unwanted time. It is highly recommended to
use an explicit version, e.g. 'org.sonarsource.scanner.maven:sonar-maven-plugin:4.0.0.4121'.
[INFO] 22:34:22.632 Java 24.0.1 Oracle Corporation (64-bit)
[INFO] 22:34:22.632 Windows 10 10.0 (amd64)
[INFO] 22:34:22.771 User cache: C:\Users\Usuario\.sonar\cache
[INFO] 22:34:22.821 Communicating with SonarQube Server 9.9.8.100196
[INFO] 22:34:23.835 Default locale: 'es_PE', source code encoding: 'UTF-8'
[INFO] 22:34:23.257 Load global settings
[INFO] 22:34:23.379 Load global settings (done) | time=122ms
[INFO] 22:34:23.381 Server id: 147B411E-AZn6ekjYDnXn3WCigyQ7
[INFO] 22:34:23.384 User cache: C:\Users\Usuario\.sonar\cache
[INFO] 22:34:23.387 Load/download plugins
[INFO] 22:34:23.387 Load plugins index
[INFO] 22:34:23.491 Load plugins index (done) | time=104ms
[INFO] 22:34:23.528 Load/download plugins (done) | time=141ms
[INFO] 22:34:23.867 Process project properties
[INFO] 22:34:23.874 Process project properties (done) | time=7ms
[INFO] 22:34:23.877 Execute project builders
[INFO] 22:34:23.879 Execute project builders (done) | time=2ms
[INFO] 22:34:23.883 Project key: com.example:quality-lab-demo-backend
[INFO] 22:34:23.883 Base dir: D:\quality-lab-demo\backend
[INFO] 22:34:23.883 Working dir: D:\quality-lab-demo\backend\target\sonar
[INFO] 22:34:23.889 Load project settings for component key: 'com.example:quality-lab-demo-backend'
[INFO] 22:34:23.992 Load project settings for component key: 'com.example:quality-lab-demo-backend' (done) | time=103ms
[INFO] 22:34:24.120 Load quality profiles
[INFO] 22:34:24.241 Load quality profiles (done) | time=121ms
[INFO] 22:34:24.252 Load active rules
[INFO] 22:34:27.362 Load active rules (done) | time=3110ms
[INFO] 22:34:27.365 Load analysis cache
[INFO] 22:34:27.456 Load analysis cache (300 bytes) | time=91ms
[WARNING] 22:34:27.482 Property 'sonar.password' is deprecated. It will not be supported in the future. Please instead use the 'sonar.login' parameter with a token.
[INFO] 22:34:27.491 Load project repositories
[INFO] 22:34:27.582 Load project repositories (done) | time=91ms
[INFO] 22:34:27.604 Indexing files...
[INFO] 22:34:27.604 Project configuration:
[INFO] 22:34:27.741 11 files indexed
[INFO] 22:34:27.741 0 files ignored because of scm ignore settings
[INFO] 22:34:27.742 Quality profile for java: Sonar way
[INFO] 22:34:27.743 Quality profile for xml: Sonar way
[INFO] 22:34:27.743 ----- Run sensors on module quality-lab-demo-backend
[INFO] 22:34:27.819 Load metrics repository
[INFO] 22:34:27.915 Load metrics repository (done) | time=96ms
[INFO] 22:34:28.904 Sensor JavaSensor [java]
[INFO] 22:34:28.908 Configured Java source version (sonar.java.source): 17
[INFO] 22:34:28.911 JavaClasspath initialization
[INFO] 22:34:28.915 JavaClasspath initialization (done) | time=4ms
[INFO] 22:34:28.916 JavaTestClasspath initialization
[INFO] 22:34:28.922 JavaTestClasspath initialization (done) | time=6ms
[INFO] 22:34:28.930 Server-side caching is enabled. The Java analyzer will not try to leverage data from a previous analysis.
[INFO] 22:34:28.933 Using ECJ batch to parse 7 Main java source files with batch size 421 KB.
[INFO] 22:34:29.071 Starting batch processing.
[INFO] 22:34:29.460 The Java analyzer cannot skip unchanged files in this context. A full analysis is performed for all files.
[INFO] 22:34:29.900 100% analyzed
[INFO] 22:34:29.901 Batch processing: Done.
[INFO] 22:34:29.902 Did not optimize analysis for any files, performed a full analysis for all 7 files.
[INFO] 22:34:29.904 Using ECJ batch to parse 3 Test java source files with batch size 421 KB.
```

```
MINGW64/d/quality-lab-demo/backend
[INFO] 22:34:27.741 11 files indexed
[INFO] 22:34:27.741 0 files ignored because of scm ignore settings
[INFO] 22:34:27.742 Quality profile for java: Sonar way
[INFO] 22:34:27.743 Quality profile for xml: Sonar way
[INFO] 22:34:27.743 ----- Run sensors on module quality-lab-demo-backend
[INFO] 22:34:27.819 Load metrics repository
[INFO] 22:34:27.915 Load metrics repository (done) | time=96ms
[INFO] 22:34:28.904 Sensor JavaSensor [java]
[INFO] 22:34:28.908 Configured Java source version (sonar.java.source): 17
[INFO] 22:34:28.911 JavaClasspath initialization
[INFO] 22:34:28.915 JavaClasspath initialization (done) | time=4ms
[INFO] 22:34:28.916 JavaTestClasspath initialization
[INFO] 22:34:28.922 JavaTestClasspath initialization (done) | time=6ms
[INFO] 22:34:28.930 Server-side caching is enabled. The Java analyzer will not try to leverage data from a previous analysis.
[INFO] 22:34:28.933 Using ECJ batch to parse 7 Main java source files with batch size 421 KB.
[INFO] 22:34:29.071 Starting batch processing.
[INFO] 22:34:29.460 The Java analyzer cannot skip unchanged files in this context. A full analysis is performed for all files.
[INFO] 22:34:29.900 100% analyzed
[INFO] 22:34:29.901 Batch processing: Done.
[INFO] 22:34:29.902 Did not optimize analysis for any files, performed a full analysis for all 7 files.
[INFO] 22:34:29.904 Using ECJ batch to parse 3 Test java source files with batch size 421 KB.
[INFO] 22:34:29.909 Starting batch processing.
[INFO] 22:34:30.017 100% analyzed
[INFO] 22:34:30.017 Batch processing: Done.
[INFO] 22:34:30.019 Did not optimize analysis for any files, performed a full analysis for all 3 files.
[INFO] 22:34:30.019 No "Generated" source files to scan.
[INFO] 22:34:30.019 Sensor JavaSensor [java] (done) | time=115ms
[INFO] 22:34:30.019 Sensor JaCoCo XML Report Importer [jacoco]
[INFO] 22:34:30.021 'sonar.coverage.jacoco.xmlReportPaths' is not defined. Using default locations: target/site/jacoco/jacoco.xml,target/site/jacoco-it/jacoco.xml,build/reports/ja
coco/test/jacoco/jacoco.xml
[INFO] 22:34:30.021 No report imported, no coverage information will be imported by JaCoCo XML Report Importer
[INFO] 22:34:30.022 Sensor JaCoCo XML Report Importer [jacoco] (done) | time=3ms
[INFO] 22:34:30.022 Sensor CSS Rules [javascript]
[INFO] 22:34:30.022 No CSS, PHP, HTML or VueJS files are found in the project. CSS analysis is skipped.
[INFO] 22:34:30.022 Sensor CSS Rules [javascript] (done) | time=0ms
[INFO] 22:34:30.023 Sensor C# Project Type Information [csharp]
[INFO] 22:34:30.023 Sensor C# Project Type Information [csharp] (done) | time=0ms
[INFO] 22:34:30.023 Sensor C# Analysis Log [csharp]
[INFO] 22:34:30.032 Sensor C# Analysis Log [csharp] (done) | time=9ms
[INFO] 22:34:30.032 Sensor C# Properties [csharp]
[INFO] 22:34:30.033 Sensor C# Properties [csharp] (done) | time=0ms
[INFO] 22:34:30.033 Sensor SurefireSensor [java]
[INFO] 22:34:30.033 parsing [D:/quality-lab-demo/backend/target/surefire-reports]
[INFO] 22:34:30.035 Sensor SurefireSensor [java] (done) | time=2ms
[INFO] 22:34:30.035 Sensor HTML [web]
[INFO] 22:34:30.037 Sensor HTML [web] (done) | time=2ms
[INFO] 22:34:30.037 Sensor XML Sensor [xml]
[INFO] 22:34:30.039 1 source file to be analyzed
[INFO] 22:34:30.145 1/1 source file has been analyzed
[INFO] 22:34:30.145 Sensor XML Sensor [xml] (done) | time=108ms
[INFO] 22:34:30.146 Sensor TextAndSecretsSensor [text]
[INFO] 22:34:30.151 1 source files to be analyzed
[INFO] 22:34:30.163 11/11 source files have been analyzed
[INFO] 22:34:30.163 Sensor TextAndSecretsSensor [text] (done) | time=17ms
[INFO] 22:34:30.164 Sensor VB.NET Project Type Information [vbnet]
[INFO] 22:34:30.164 Sensor VB.NET Project Type Information [vbnet] (done) | time=0ms
[INFO] 22:34:30.165 Sensor VB.NET Analysis Log [vbnet]
[INFO] 22:34:30.179 Sensor VB.NET Analysis Log [vbnet] (done) | time=14ms
[INFO] 22:34:30.179 Sensor VB.NET Properties [vbnet]
[INFO] 22:34:30.179 Sensor VB.NET Properties [vbnet] (done) | time=0ms
[INFO] 22:34:30.180 Sensor IaC Docker Sensor [iac]
[INFO] 22:34:30.181 0 source files to be analyzed
[INFO] 22:34:30.217 0/0 source files have been analyzed
[INFO] 22:34:30.217 Sensor IaC Docker Sensor [iac] (done) | time=37ms
[INFO] 22:34:30.221 ----- Run sensors on project
[INFO] 22:34:30.261 Sensor Analysis Warnings import [csharp]
[INFO] 22:34:30.262 Sensor Analysis Warnings import [csharp] (done) | time=1ms
[INFO] 22:34:30.262 Sensor Zero Coverage Sensor
[INFO] 22:34:30.268 Sensor Zero Coverage Sensor (done) | time=6ms
[INFO] 22:34:30.268 Sensor Java CPD Block Indexer
[INFO] 22:34:30.281 Sensor Java CPD Block Indexer (done) | time=13ms
[INFO] 22:34:30.290 SCM Publisher SCM provider for this project is: git
[INFO] 22:34:30.292 SCM Publisher 2 source files to be analyzed
[INFO] 22:34:30.567 SCM Publisher 0/2 source files have been analyzed (done) | time=275ms
[WARNING] 22:34:30.567 Missing blame information for the following files:
[WARNING] 22:34:30.568   * src/main/java/com/example/qualitydemo/controller/HealthController.java
[WARNING] 22:34:30.568   * pom.xml
[WARNING] 22:34:30.568 This may lead to missing/broken Features in SonarQube
[INFO] 22:34:30.571 CPD Executor 4 files had no CPD blocks
[INFO] 22:34:30.571 CPD Executor Calculating CPD for 3 files
[INFO] 22:34:30.577 CPD Executor CPD calculation finished (done) | time=6ms
[INFO] 22:34:30.633 Analysis report generated in 52ms, dir size=147.8 kB
[INFO] 22:34:30.807 Analysis report compressed in 174ms, zip size=35.3 kB
[INFO] 22:34:30.918 Analysis report uploaded in 110ms
[INFO] 22:34:30.920 ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=com.example%3Aquality-lab-demo-backend
[INFO] 22:34:30.920 Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] 22:34:30.920 More about the report processing at http://localhost:9000/api/ce/task?id=AZn6iHFrDkn3Wcig8KX
[INFO] 22:34:30.932 Analysis total time: 7.342 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.773 s
[INFO] Finished at: 2025-10-18T22:34:30-05:00
[INFO] -----
Usuario@DESKTOP-9HHSJTO MINGW64 /d/quality-lab-demo/backend (main)
$ |
```

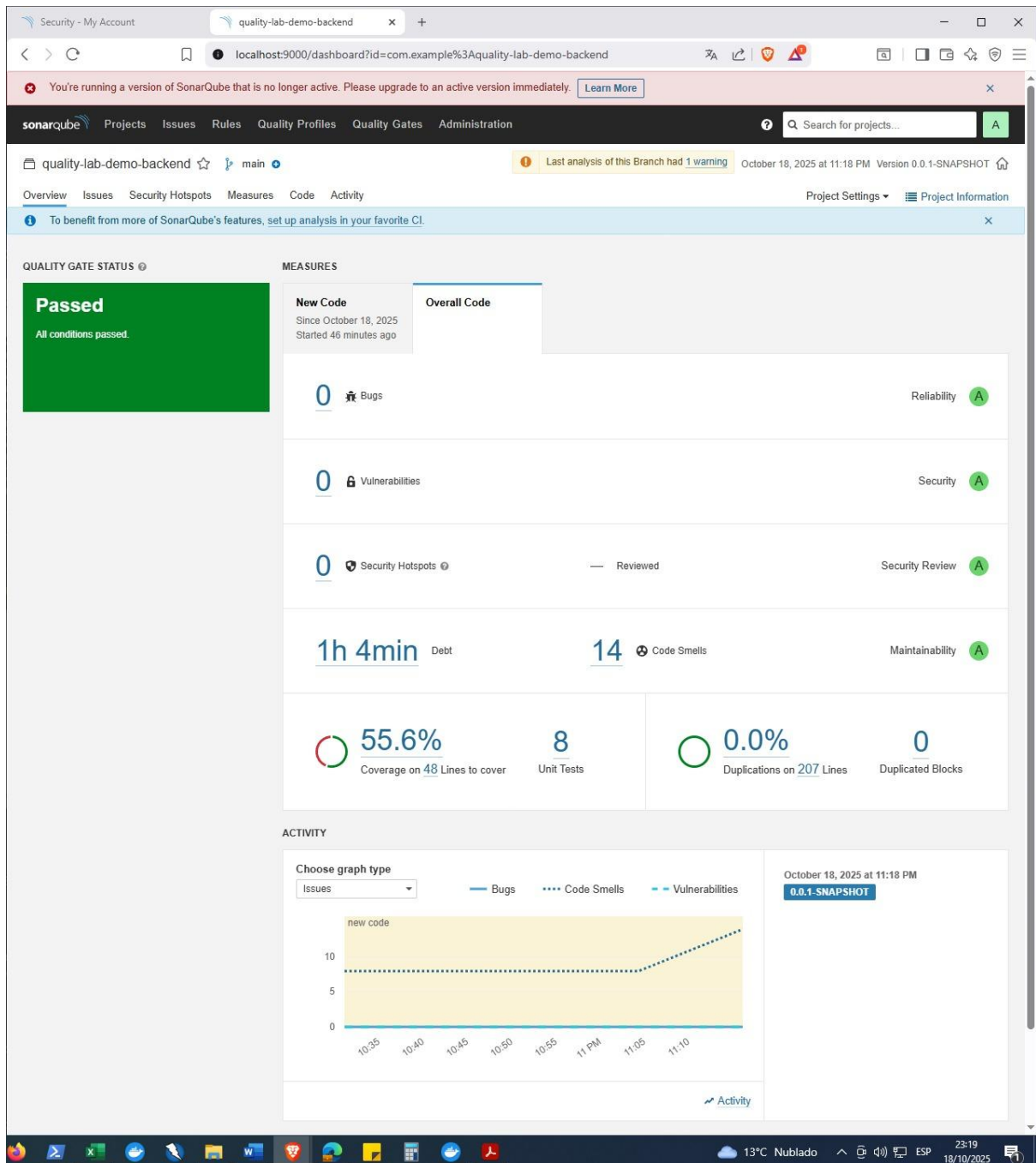
(Pantallazo 2.3 - Reporte de SONARQUBE):

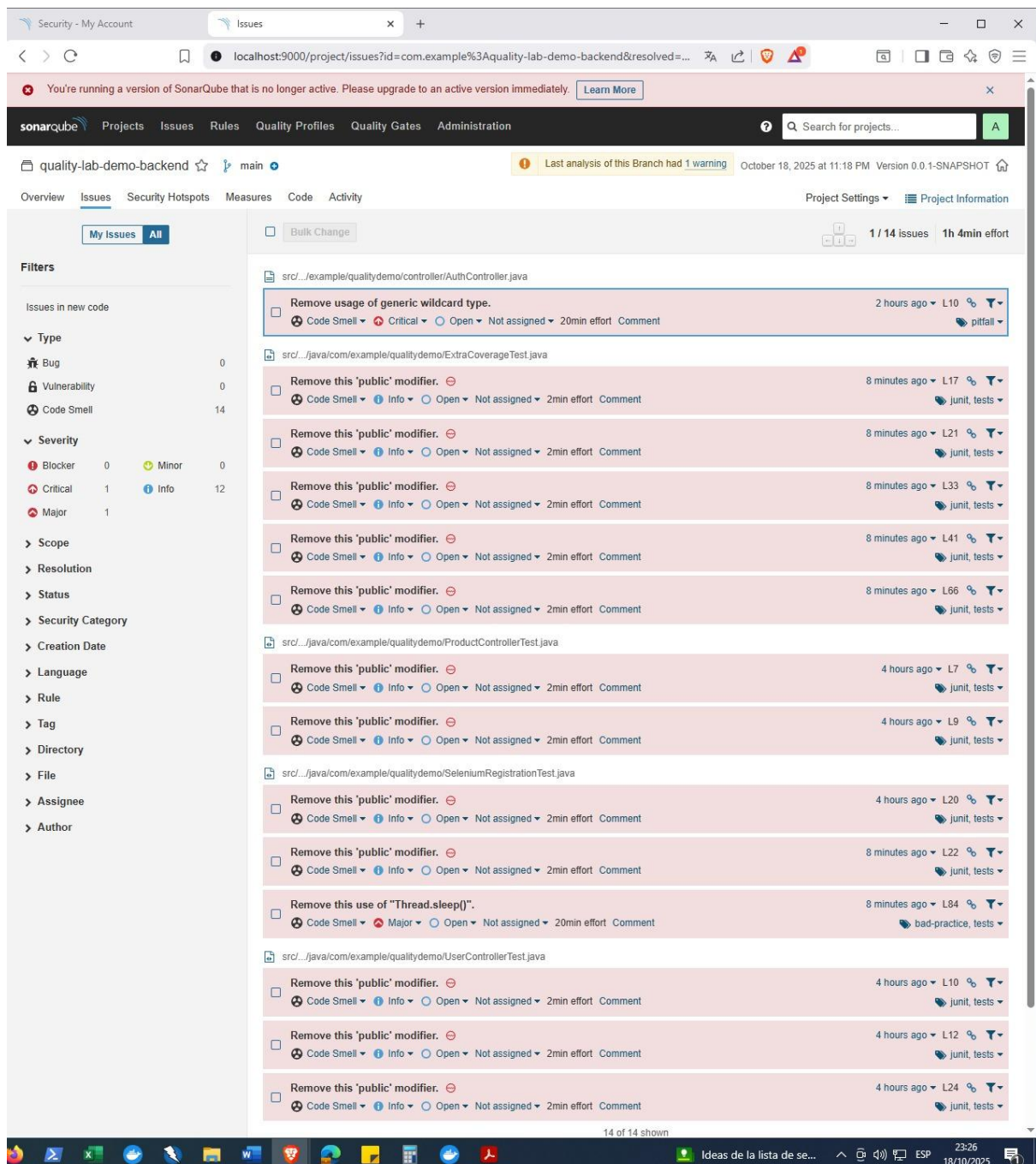
Al iniciar SonarQube, se cargan plugins, se conecta a la base de datos, y se habilita el panel web donde se visualizan métricas como duplicación, cobertura de pruebas, complejidad y vulnerabilidades. Integrarlo con proyectos Maven o Gradle permite automatizar el análisis en cada compilación, mejorando la mantenibilidad y seguridad del software.



(Pantallazo 2.4 - Reporte de SONARQUBE):

El dashboard de SonarQube muestra un análisis exitoso con una cobertura de código del 55.6% sobre 48 líneas analizadas, donde se ejecutaron 8 pruebas unitarias. El proyecto ha pasado el Quality Gate satisfactoriamente, indicando que cumple con los estándares de calidad establecidos. Se identificaron 14 code smells que representan aproximadamente 1 hora y 4 minutos de deuda técnica. El gráfico de actividad muestra el análisis de "nuevo código" realizado el 18 de octubre de 2025, confirmando que el pipeline está funcionando correctamente y generando métricas accionables para el equipo de desarrollo.





(Pantallazo 2.5 - Reporte de SONARQUBE):

El análisis detallado revela una distribución desigual de la cobertura: mientras que PageController.java, ProductController.java, UserController.java y User.java alcanzan un 100% de cobertura (indicando pruebas unitarias completas), tres archivos críticos presentan 0% de cobertura: AuthController.java (6 líneas sin cubrir), HealthController.java (11 líneas sin cubrir) y QualityLabDemoApplication.java (3 líneas sin cubrir). Esta distribución identifica claramente los puntos ciegos en las pruebas, especialmente preocupante en AuthController

que maneja funcionalidades de autenticación sensibles. El análisis cubre 7 archivos en total, proporcionando una visión completa del estado de la calidad del código en el proyecto.

Security - My Account

Coverage

localhost:9000/component_measures?metric=coverage&view=list&tid=com.example%3Aq...

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.

Learn More

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality GatesAdministration

Search for projects...

quality-lab-demo-backend

main

Last analysis of this Branch had 1 warning

October 18, 2025 at 11:18 PM

Version 0.0.1-SNAPSHOT

OverviewIssuesSecurity HotspotsMeasuresCodeActivity

Project SettingsProject Information

Project Overview

Reliability

Security

Security Review

Maintainability

CoverageCOVERAGE

Overview

On new code

Lines to Cover0

Uncovered Lines0

Conditions to Cover0

Uncovered Conditions0

Overall

Coverage55.6%

Lines to Cover48

Uncovered Lines20

Line Coverage58.3%

Conditions to Cover6

Uncovered Conditions4

Condition Coverage33.3%

Tests

Unit Tests8

Errors0

Failures0

Skipped0

Success100%

Duration3s

Duplications

Size

Complexity

Issues

quality-lab-demo-backend

View asList

to select files

to navigate

7 files

Coverage55.6%

New Code: Since October 18, 2025

	Coverage	Uncovered Lines	Uncovered Conditions
src/main/java/com/example/qualitydemo/controller/AuthController.java	0.0%	6	4
src/main/java/com/example/qualitydemo/controller/HealthController.java	0.0%	11	-
src/main/java/com/example/qualitydemo/QualityLabDemoApplication.java	0.0%	3	-
src/main/java/com/example/qualitydemo/controller/PageController.java	100%	0	-
src/main/java/com/example/qualitydemo/controller/ProductController.java	100%	0	-
src/main/java/com/example/qualitydemo/model/User.java	100%	0	-
src/main/java/com/example/qualitydemo/controller/UserController.java	100%	0	0

7 of 7 shown

Embedded database should be used for evaluation purposes only

The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

SonarQube™ technology is powered by SonarSource SA

Community Edition - v9.9.8 (build 100196)

NO LONGER ACTIVE

LGPL v3

Community

Documentation

Plugins

Web API

13°C

Nublado

23:34

18/10/2025